

DATA ENGINEERING GUIDE

Medallion Architecture

A layered approach to building reliable, scalable data pipelines with
Bronze, Silver & Gold zones

Bronze — Raw, unprocessed ingestion

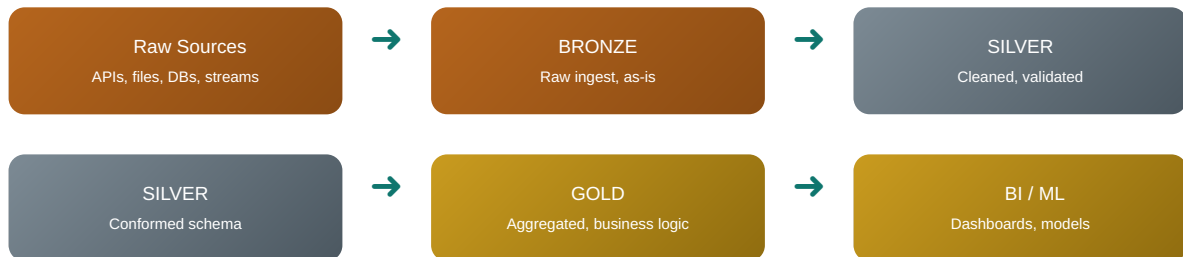
Silver — Cleaned & conformed data

Gold — Business-ready aggregates

1. What Is Medallion Architecture?

Medallion architecture is a data design pattern used in modern lakehouses to logically organize data into three progressive layers of quality and refinement: **Bronze**, **Silver**, and **Gold**. Popularized by Databricks, the pattern incrementally improves data structure, quality, and trustworthiness as it flows through each layer.

Instead of building one monolithic pipeline that goes straight from raw source to polished report, medallion architecture breaks the transformation into clear, auditable stages. Each layer has a distinct purpose, its own storage location, and its own set of consumers.



Why "Medallion"?

The name comes from the idea of data "leveling up" in quality at each stage — much like a medal progresses from bronze to silver to gold. It has nothing to do with the data itself; it's purely a metaphor for increasing refinement and trust.

Core Principles

- **Incremental refinement** — each layer improves on the one before it; no layer skips quality steps.
- **Traceability** — raw data is always preserved in Bronze, so you can always reprocess from source.
- **Separation of concerns** — ingestion, cleaning, and business logic live in different layers, making pipelines easier to debug and maintain.
- **Reusability** — Silver-layer data can feed many different Gold-layer models without re-cleaning raw data each time.
- **Schema evolution** — Bronze tolerates messy/changing schemas; Silver and Gold enforce increasingly strict contracts.

Where It's Used

Medallion architecture is the default pattern in Databricks Lakehouse, and is commonly implemented on Delta Lake, Apache Iceberg, Apache Hudi, or plain Parquet on S3/HDFS/ADLS — orchestrated with Spark, Airflow, or dbt.

2. Bronze Layer — Raw Ingestion

Bronze Layer

The landing zone for raw data, ingested **as-is** from source systems with minimal or no transformation. It acts as the single source of truth and historical archive.

Characteristics

- Data is stored in its **original format** (or converted to Parquet/Delta for storage efficiency, without altering values).
- **Append-only** — new data is added, existing records are rarely updated or deleted.
- Includes **metadata columns**: ingestion timestamp, source system, batch/file ID.
- Schema is **loosely enforced** — Bronze should accept whatever the source sends, including malformed rows.
- Serves as the **replay source** — if downstream logic changes, you can reprocess Silver/Gold from Bronze without re-fetching from the original system.

Typical Sources

Source Type	Examples	Ingestion Pattern
Batch files	CSV, JSON, XML dumps	Scheduled full/incremental load
Databases	MySQL, PostgreSQL, SQL Server (CDC)	Change Data Capture / JDBC extract
Streaming	Kafka, Kinesis, Event Hubs	Structured Streaming / micro-batches
APIs	REST/GraphQL third-party APIs	Scheduled pulls, webhook drops

Example: Bronze Ingestion in PySpark

PYSPARK

```
from pyspark.sql.functions import current_timestamp, input_file_name

# Read raw CSV files exactly as they arrive - no cleaning yet
bronze_df = (
    spark.read
        .option("header", True)
        .option("inferSchema", False) # keep everything as string in Bronze
        .csv("/mnt/raw/sales/*.csv")
)

# Add ingestion metadata - critical for auditability
bronze_df = (
    bronze_df
        .withColumn("_ingested_at", current_timestamp())
        .withColumn("_source_file", input_file_name())
)

# Write as-is, append-only, partitioned by ingestion date
(
    bronze_df.write
        .format("delta")
        .mode("append")
        .partitionBy("_ingestion_date")
        .save("/mnt/bronze/sales")
)
```

Common Mistake

Don't clean or filter data in Bronze. Dropping "bad" rows here means you lose the ability to audit or reprocess them later. Bronze should be a faithful, complete copy of what the source sent.

3. Silver Layer — Cleaned & Conformed

Silver Layer

Bronze data is **cleaned, validated, deduplicated, and conformed** into a consistent schema. This is the "single version of truth" layer that most downstream teams query directly.

What Happens in Silver

- **Schema enforcement** — correct data types, column names, and structure are applied.
- **Data quality checks** — nulls handled, duplicates removed, invalid rows quarantined or fixed.
- **Deduplication** — using primary keys, watermarks, or CDC merge logic.
- **Joins across sources** — Silver often merges Bronze tables from multiple systems into unified entities (e.g., combining CRM and billing customer records).
- **Standardization** — consistent units, date formats, currency, casing, and naming conventions.
- Still **grain-preserving** — Silver is typically at the same granularity as the source (one row per transaction, per event, etc.), not yet aggregated.

Example: Silver Transformation in PySpark

PYSPARK

```
from pyspark.sql.functions import col, trim, upper, to_date
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number

bronze_df = spark.read.format("delta").load("/mnt/bronze/sales")

silver_df = (
    bronze_df
    # Enforce correct types
    .withColumn("sale_amount", col("sale_amount").cast("double"))
    .withColumn("sale_date", to_date(col("sale_date"), "yyyy-MM-dd"))
    # Standardize text fields
    .withColumn("customer_name", trim(upper(col("customer_name"))))
    # Drop rows with missing critical fields
    .filter(col("sale_amount").isNotNull() & col("sale_date").isNotNull())
)

# Deduplicate: keep the latest record per transaction_id
window_spec = Window.partitionBy("transaction_id").orderBy(col("_ingested_at").desc())

silver_df = (
    silver_df
    .withColumn("_row_num", row_number().over(window_spec))
    .filter(col("_row_num") == 1)
    .drop("_row_num")
)

(
    silver_df.write
    .format("delta")
    .mode("overwrite")
    .option("mergeSchema", "true")
    .save("/mnt/silver/sales")
)
```

Use **MERGE** for Incremental Silver Loads

In production, Silver tables are usually updated with a `MERGE INTO` (upsert) statement rather than a full overwrite, so you only reprocess new or changed Bronze records.

4. Gold Layer — Business-Level Aggregates

Gold Layer

Highly refined, business-curated data — **aggregated, denormalized, and modeled** specifically for reporting, dashboards, and machine learning. This is what analysts and BI tools consume directly.

What Happens in Gold

- **Aggregation** — daily/monthly rollups, KPIs, summary metrics.
- **Dimensional modeling** — star schemas with fact and dimension tables (e.g., fact_sales, dim_customers, dim_products).
- **Business logic applied** — revenue calculations, churn definitions, customer segments.
- **Optimized for read performance** — denormalized, partitioned, and indexed for fast BI queries.
- Consumed by **Power BI, Tableau, Looker**, ML feature stores, and executive dashboards.

Example: Gold Aggregation in PySpark

PYSPARK

```
from pyspark.sql.functions import sum as _sum, count, avg, date_trunc

silver_df = spark.read.format("delta").load("/mnt/silver/sales")

gold_monthly_revenue = (
    silver_df
    .withColumn("month", date_trunc("month", col("sale_date")))
    .groupBy("month", "region")
    .agg(
        _sum("sale_amount").alias("total_revenue"),
        count("transaction_id").alias("num_transactions"),
        avg("sale_amount").alias("avg_order_value"),
    )
    .orderBy("month", "region")
)

(
    gold_monthly_revenue.write
    .format("delta")
    .mode("overwrite")
    .save("/mnt/gold/monthly_revenue_by_region")
)
```

Gold Is Purpose-Built

Unlike Silver (one general-purpose "clean" table), it's normal to have **many different Gold tables** — each shaped for a specific report, dashboard, or ML use case, all built from the same trusted Silver layer.

5. Side-by-Side Comparison

Aspect	BRONZE	SILVER	GOLD
Purpose	Raw archive / audit trail	Clean, trusted, queryable data	Business-ready analytics
Data Quality	As-is, may contain errors	Validated, deduplicated	Curated, business-logic applied
Schema	Loose / source-defined	Enforced, conformed	Dimensional (star schema)
Granularity	Row-level, as ingested	Row-level, deduplicated	Aggregated / summarized
Write Pattern	Append-only	Upsert / merge	Overwrite / incremental refresh
Typical Consumers	Data engineers	Data analysts, data scientists	BI tools, executives, ML models
Reprocessing	N/A — source of truth	Rebuilt from Bronze if needed	Rebuilt from Silver if needed

6. Benefits, Pitfalls & Best Practices

Benefits

- Clear separation of ingestion vs. transformation vs. consumption logic.

Common Pitfalls

- Full historical replay capability from Bronze.
- Easier debugging — you can inspect data at each layer
- **Skipping Bronze** — you can inspect data at each layer
- **Wide dependency** — data directly without keeping a raw copy makes it impossible to debug logic later.
- Multiple Gold tables can be derived from one trusted Silver source.
- Supports both batch and streaming with the same layered pattern.
- **Skewed growth** — tables evolve with heaps growth in different spaces with different layers.

Over-aggregating in Silver

Silver should stay close to source granularity. Premature aggregation in Silver limits flexibility for the many different Gold tables that might need to be built later.

No data quality checks

Without validation rules in Silver (nulls, duplicates, referential integrity), bad data silently flows into Gold and corrupts dashboards and models.

Best Practices Checklist

- ☒ Use a lakehouse table format (Delta Lake, Iceberg, or Hudi) for ACID transactions and time travel.
- ☒ Partition Bronze by ingestion date; partition Gold by the dimensions most queried (e.g., month, region).
- ☒ Track lineage: know which Bronze records fed which Silver and Gold rows.
- ☒ Automate data quality checks (e.g., Great Expectations, Delta Live Tables expectations) between layers.
- ☒ Document schema contracts for Silver and Gold so downstream consumers aren't broken by changes.
- ☒ Use incremental processing (CDC, watermarks, MERGE) rather than full reprocessing where possible.

7. Quick Reference Summary

Layer	One-Line Definition
BRONZE	Raw, unaltered data landed exactly as received from source systems.
SILVER	Validated, deduplicated, and conformed data at source granularity.
GOLD	Aggregated, business-modeled data ready for BI and ML consumption.

Remember

Data quality and business value increase as data moves from Bronze → Silver → Gold, while raw fidelity and reprocessing flexibility are always preserved by keeping Bronze untouched.

End of guide — Medallion Architecture: Bronze, Silver & Gold Layers.